

Práctica 3-A: Programación con GPU/NVidia Cuda

En este ejercicio se pide poner en práctica el conocimiento adquirido sobre programación con tarjetas gráficas usando el SDK de NVidia Cuda. Los siguientes ejercicios están pensados para dar una introducción al lenguaje de programación y sacar provecho del paralelismo en tarjetas gráficas.

Ejercicio:

Se pide implementar una versión GPU del ejercicio pedido en la práctica 1. Se recuerdan los detalles de esa práctica:

- Crear un programa “main” que cree tres arrays de 1000 matrices llamadas A1, A2 y AResultados.
- Rellenar los datos de los arrays A1 y A2 con datos random.
- Crear una función llamada “multiplicaMatrices_tradicional”, que reciba por parámetros los arrays A1, A2, AResultados y el tamaño de los mismos. Dentro de esta función, se debe llamar en un bucle al método “multMatrix_tradicional”, y tomar sus tiempos de ejecución total para calcular la multiplicación de las mil matrices de A1 por las mil matrices de A2, y guardar los resultados en el array AResultados. Es decir, mil multiplicaciones en total:
 - o Por cada matriz i de A1 y A2
 - ❓ AResultados[i]= multMatrix_tradicional (A1[i],A2[i]);
- Anotar el tiempo que tarda en realizar la operación anterior. El tiempo obtenido se usará como referencia para el resto de implementaciones.

Parte 1: Implementación GPU de multiplicación de matrices

- Crear un array extra llamado “AResultadosGPU_h”, de 1000000 de matrices cada uno. Los arrays con sufijos “_d” son arrays de GPU para las funciones de cuda.

- Se copiarán los datos de matrices generados en CPU a los arrays de GPUs , y al finalizar la ejecución se copiarán igualmente los datos resultado a los arrays de CPU/Host.
- Modificar la cabecera de la función “multMatrix_tradicional” para que sea de tipo “__host__” y “__device__”. Se usará sin modificar desde las funciones __global__ de GPU para multiplicar los datos.
- Crear una función llamada “multiplicaMatricesGPU” , que reciba por parámetros los arrays A1, A2, AResultadosGPU y el tamaño de los arrays.
- Crear una función de tipo __global__ llamada “multiplicaMatricesGPU_kernel”, la cual recibe los arrays A1_d, A2_d, AResultadosGPU_d y el tamaño de los arrays. Esta función calculará la multiplicación de una de las matrices de A1 por A2, y guardará el resultado en una posición de AResultadosGPU. LLamará internamente a la versión “__device__” de “multMatrix_tradicional”, pasándole una única matriz, cuyos índices deben ser calculados a partir de los identificadores de thread/bloque.
- La función “multiplicaMatricesGPU” creará los arrays “A1_d”, “A2_d” y “AResultadosGPU_d” , en los cuales almacenará una copia de los datos recibidos (sólo son necesarios los arrays A1 y A2), calculará el número de bloques y thread por bloque óptimo, y llamará a la función __global__ “multiplicaMatricesGPU_kernel” con los datos de GPU. Al acabar la ejecución, guardará una copia de los resultados en el array “AResultadosGPU_h” y liberará los arrays temporales de GPU.
- Anotar el tiempo que tarda en realizar la operación anterior y calcular “speedup”:
 - o Dividir el tiempo que tardó “multiplicaMatrices_tradicional” entre el tiempo obtenido.
 - o Si es >1 quiere decir que hay speedup positivo, es más rápido con threads
 - o Si es <1 quiere decir que hay speedup negativo es más rápido Tradicional
- Hacer pruebas con distintas configuraciones de bloque/thread por bloque para encontrar la configuración óptima.
- Repetir varias veces, la primera vez que se ejecuta un programa de Cuda es probable que sea la más lenta (con diferencia).

Entrega:

Entregar el código desarrollado por el alumno en blackboard antes de la fecha indicada. Los archivos de código entregados deberán tener el nombre del alumno en un comentario dentro del código. El archivo entregado deberá tener la siguiente nomenclatura:

- PBN_PR3A_NOMBRE_APELLIDO.zip

Cualquier archivo entregado que no se atenga a estas reglas de entrega podrá ser ignorado.